

Летучка 5 · 7 минут · Ваше ФИ: _____

1. Расставьте скобки в типе `String -> Int -> Bool`: _____
2. Напишите наиболее общий тип `\x -> (x, x)`: _____
3. Замените скобки на \$, где это возможно: `f (g x) (h y) ≡` _____
4. Дано `fac 0 = 1; fac n = n * fac (n - 1)`. Один шаг без арифметики: `fac 3 ⇝` _____
5. Список `[10, 20, 30]` сопоставили с образцом `x : xs`. Чему равно `xs`? _____

✂ _____

Листок 5. Типы данных

Задание 1 · слайд 4. Напишите типы `Dog` и `Parrot`.

```
data CatT = CatD String Int
data Pet = Cat CatT | Dog String Int | Parrot
```

Задание 2 · слайд 7. Закончите функцию, чтобы она обрабатывала каждый вариант `Pet`. Возраст попугая в нашем типе не хранится.

```
age :: Pet -> Maybe Int
age pet = case pet of
  Dog _ age -> Just age
  Cat (CatD _ age) -> Just age
  -- допишите недостающую ветку
```

Задание 3 · слайд 10. Прорецируйте `height t` до числа. Высоту считаем в вершинах: у листа она равна 1.

```
data Tree a = Leaf a | Node (Tree a) a (Tree a)
```

```
t :: Tree Int
t = Node (Leaf 1) 2 (Node (Leaf 3) 4 (Leaf 5))
```

```
height :: Tree a -> Int
height t = case t of
  Leaf _ -> 1
  Node l _ r -> 1 + max (height l) (height r)
```

Задание 4 • слайд 12. Напишите `sumTree :: Tree Int -> Int` целиком. Функция должна складывать все значения дерева: и в листьях, и во внутренних узлах.

Задание 5 • слайд 15. Заполните пропуски. Функция `f` может менять тип содержимого, а форма дерева должна сохраниться.

```
transform :: (a -> b) -> Tree a -> Tree b
transform f t = case t of
  Leaf x -> Leaf _____
  Node l x r -> Node _____
```

Задание 6 • слайд 22. Напишите безопасную функцию взятия головы списка: на пустом списке возвращайте `Nothing`.

```
safeHead :: [a] -> Maybe a
safeHead xs = case xs of
  [] -> _____
  x : _ -> _____
```

Задание 7 • слайд 26. Напишите `dupEvens :: [a] -> [a]`, дублирующую элементы с индексами 0, 2, 4, ... Для `[10,20,30,40]` результат — `[10,10,20,30,30,40]`. Используйте `zip`, `map` и `concat`.

Задание 8 • слайд 32. Утилита печатает справку без параметров или запускается с обязательным форматом `Plain/JSON` и выводом в `stdout` либо файл.

```
data Mode = HelpMode | RunMode
data Format = Plain | JSON
data BadOptions = BadOptions Mode (Maybe Format) (Maybe FilePath)
```

Спроектируйте `Options`, исключая неверные состояния. Сравните с `BadOptions` число состояний для двух путей `a.txt`, `b.txt` (`Nothing` означает `stdout`).